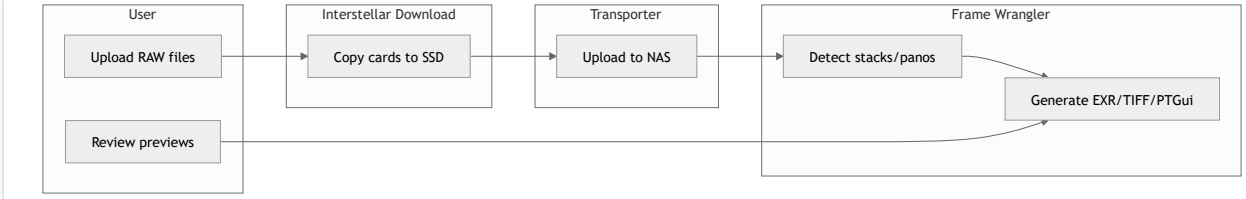
Yes — I can create swim lane diagrams in several formats depending on what you need:

- **Mermaid** (great for Markdown/docs/GitHub)
- **PlantUML**
- **ASCII/text**
- **Draw.io / diagrams.net XML**
- **SVG**
- **Visio-style structured layouts**
- **BPMN-like workflows**
- **Presentation-ready diagrams**
- **Interactive HTML/React visualizations**

For example, a simple Mermaid swim lane diagram:



Or a more formal cross-functional swimlane with:

- actors
- dependencies
- timing
- state transitions
- async queues
- retry logic
- conditional branches
- ownership boundaries

Given your existing workflows (like FrameWrangler, Interstellar Download, Transporter, and DAM automation), I can also generate:

- engineering-grade workflow specs
- pipeline orchestration diagrams
- microservice interaction diagrams
- hardware/software responsibility lanes
- camera/ESP32 event timing diagrams

If you want, give me:

1. The actors/lanes
2. The process steps
3. Any async events or decision points
4. Preferred format

...and I'll generate the diagram directly.

Sources

Okay I'm going to have to give you this information in a few different chunks but this is not related to frame wrangler this is a different system. For this diagram we have a timing controlling microcontroller which sits

between a motion activated trigger system from contraptions and the actual camera. The microcontroller has two sets of inputs and outputs One is half press the other is full press. The incoming trigger will send periodic but highly temporally unstable half press signals and will be configured to send a full press signal when a target

Show more

Got it. I'll collect the chunks first and avoid locking the diagram too early. So far I have four lanes/entities: motion trigger system, timing-control microcontroller, half-press path, full-press path, and camera.

Send the next chunk. I'll keep building the model as you go.

Okay so here's where things get complicated and I'm not quite sure how to best illustrate this on the swim lane diagram. Effectively half press will not always show up before a full press signal. The half-press signals are really just a wake up from the motion sensor that's using a PIR wide-angle sensor. For that wake up signal or what we interpret as a half press input we need to send and latch the half-press output on the camera. This will force the camera to be auto focusing with the presumption that a full press signal will be coming soon. If we do not receive a full press within 30 seconds on assert the half press on the

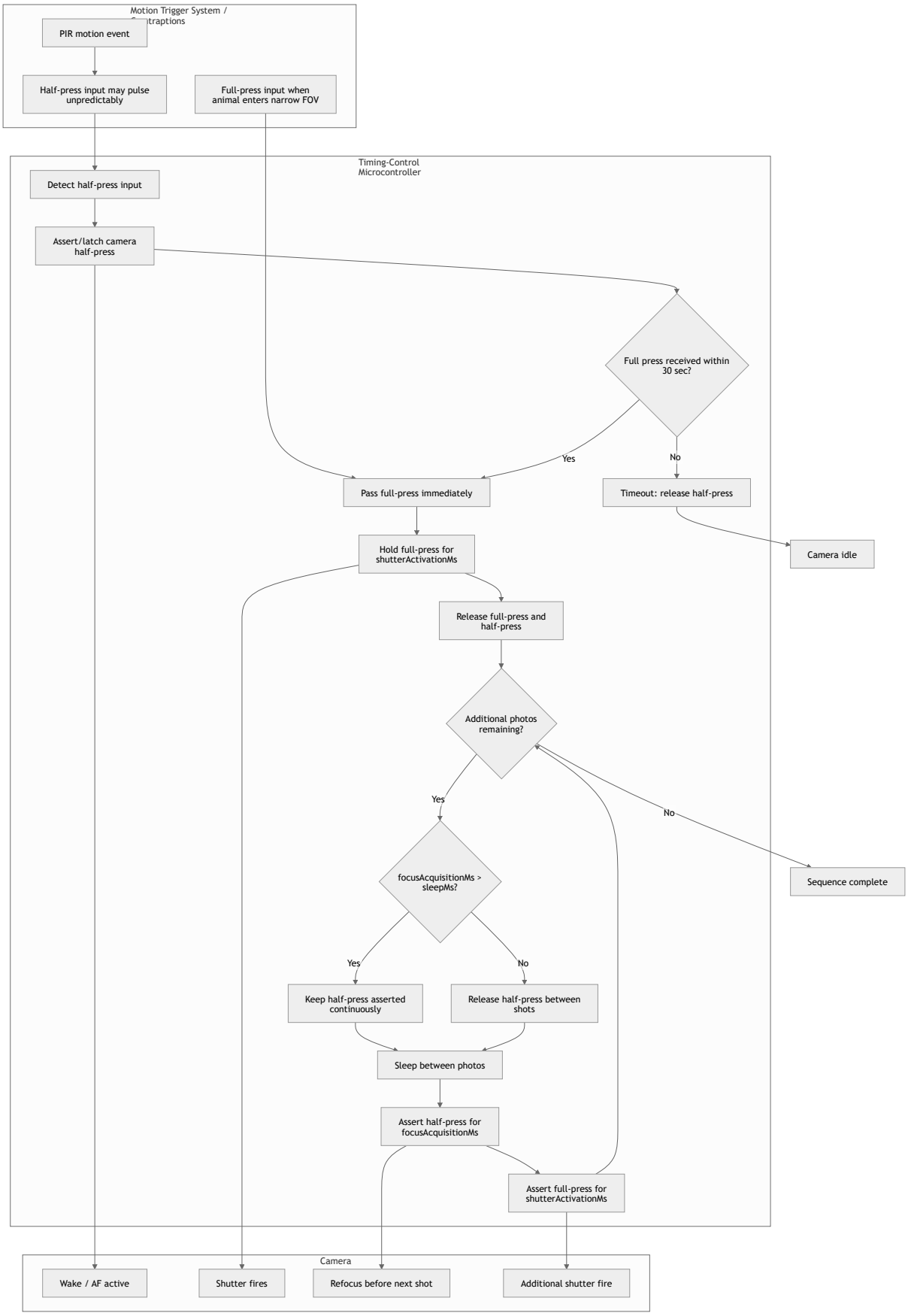
Show more

Thought for a couple of seconds

Yes — this is best shown as **two diagrams**:

1. **Swim lane / state flow** for ownership and logic
2. **Timing diagram** for exact half/full press assertions

Draft swim lane in Mermaid:


Motion Trigger System / traptions
PIR motion event
Half-press input may pulse unpredictably
Full-press input when animal enters narrow FOV
Timing-Control Microcontroller
Detect half-press input
Assert/latch camera half-press
Full press received within 30 sec?
Yes
No
Pass full-press immediately
Timeout: release half-press
Camera idle
Hold full-press for shutterActivationMs
Release full-press and half-press
Additional photos remaining?
Yes
No
Sequence complete
focusAcquisitionMs > sleepMs?
Yes
No
Keep half-press asserted continuously
Release half-press between shots
Sleep between photos
Assert half-press for focusAcquisitionMs
Assert full-press for shutterActivationMs
Camera
Wake / AF active
Shutter fires
Refocus before next shot
Additional shutter fire

And the behavior summary I'd encode in the diagram/spec:

```
Parameters:
- halfPressTimeoutMs = 30000
- shutterActivationMs = configurable
- additionalPhotoCount = X
- sleepMs = configurable
- focusAcquisitionMs = configurable

Rules:
1. Half-press input is treated as wake/prepare, not as a stable camera command.
2. On half-press input, MCU latches camera half-press ON.
3. If no full-press input arrives within 30 seconds, MCU releases camera half-press.
4. Full-press input may arrive without a prior half-press.
5. On full-press input, MCU immediately asserts camera full-press.
6. MCU holds full-press for shutterActivationMs.
7. After the initial shot, MCU releases full-press and half-press, then starts the add
8. For each additional photo:
   - wait sleepMs
   - assert half-press for focusAcquisitionMs before shutter
   - assert full-press for shutterActivationMs
9. If focusAcquisitionMs > sleepMs, half-press remains asserted continuously between s
```

For the final diagram, I'd probably label the MCU lane as a **state machine**, because that's really what this is.

ok - so -- let's put a pin in these diagrams for a moment -- I'd like to work through a set of parameters we may want for control. Here's some requirements - but we know there are additional settings we need. Let's start making a list of parameters, their default values, and what behavior they define. We'll then circle back on the diagrams and update them.

1. Use an Arduino or other tiny, low power microcontroller. It would sit in between the camera trigger receiver and the camera – i.e. inline with the cable. So it receives the

Show more

I'll turn this into a parameter table rather than a diagram for now, separating external PIR settings, MCU behavior settings, timing constants, and derived rules.

Thought for a couple of seconds

Here's a first-pass parameter list.

Parameter	Default	Units	Defines
`wakeHalfPressHoldTime`	10	sec	How long MCU holds camera HP after a brief HP/wake pulse from wide PIR
`maxWakeHalfPressHoldTime`	30	sec	Upper practical/config limit for HP latch duration
`minHalfPressBeforeShutter`	0.5	sec	Minimum HP lead time before first shutter activation
`shutterPulseDuration`	100–250	ms	How long MCU asserts full-press for each photo
`burstFrameCount`	4	frames	Total shutter activations per trigger event
`interFrameDelay`	1.0	sec	Delay between shutter pulses in MCU-generated burst
`postBurstHalfPressHoldTime`	2.0	sec	How long HP remains asserted after final frame
`triggerRetriggerPolicy`	immediate	enum	If FP arrives while HP is already active, shoot immediately
`halfPressDuringBurstPolicy`	unchanged	enum	Trigger activations do not drop or reset HP
`cameraDriveMode`	single-shot	enum	Camera stays in single-shot mode; MCU creates burst timing

Parameter	Default	Units	Defines
`widePirMode`	wake	enum	Wide PIR emits HP/wake pulse only
`farPirMode`	normal	enum	Narrow PIR emits actual shutter/full-press trigger
`pirGapTime`	5–8	sec	External PIR lockout/gap setting after a trigger
`minPirGapTime`	1	sec	Shortest expected external PIR gap
`maxPirGapTime`	60	sec	Longest expected external PIR gap
`fullPressWithoutPriorHpPolicy`	wait-if-needed	enum	If FP arrives before enough HP time, assert HP and wait up to `minHalfPressBeforeShutter`
`firstShotPolicy`	conditional-delay	enum	First frame fires instantly only if HP has already been active long enough
`sequenceRestartPolicy`	probably queue-or-ignore	enum	What to do if FP arrives during active burst; needs decision
`halfPressInputDebounce`	20–50	ms	Reject noisy/short glitches on incoming HP
`fullPressInputDebounce`	10–30	ms	Reject noisy/short glitches on incoming FP
`inputActivePolarity`	active-low	enum	Most camera trigger systems short line to ground; verify hardware
`outputDriveMode`	open-drain / opto	enum	MCU should mimic cable switch closure, not drive voltage into camera

Parameter	Default	Units	Defines
`powerSaveIdleMode`	enabled	bool	MCU sleeps when no HP/FP activity is active
`statusLedMode`	optional	enum	Disabled / debug / field-visible / field-dark

Core behavior in compact form:

Condition	Behavior
HP wake pulse received	Assert camera HP and start/refresh `wakeHalfPressHoldTime`
No FP before HP timer expires	Release camera HP
FP received and HP active for ≥ `minHalfPressBeforeShutter`	Fire first shutter immediately
FP received and HP not active long enough	Assert/keep HP, wait remaining time, then fire
Burst starts	Fire `burstFrameCount` shutter pulses
Between frames	Wait `interFrameDelay`
During burst	HP remains controlled independently; do not drop HP because of FP pulses
After final frame	Keep HP asserted for `postBurstHalfPressHoldTime`, then release unless another timer/event requires HP

The one setting I'd decide next is `sequenceRestartPolicy`:

Option	Meaning
`ignoreDuringBurst`	FP inputs during a burst are ignored
`extendAfterBurst`	FP during burst schedules another burst after current one
`restartBurstTimer`	FP during burst restarts/extends the N-frame sequence

Option	Meaning
`mergeIfHpActive`	If HP still active, new FP immediately starts another N-frame sequence after current pulse spacing allows

My default recommendation: `extendAfterBurst`, because it avoids dropped detections without creating chaotic overlapping shutter timing.

hmm, we need to make some changes to this. For example, cameraDriveMode - no idea what this does, strike it or explain what we do with that. sequenceRestartPolicy - this isn't clearly defined. It's good to have this enum - but the core behavior we want isn't one of the described values. Our target mode is something like restartBurstTimer - but it's not a /timer/ - it's a shot count. Example: If we are on shot 4 of 6, and another full press comes in - we reset that counter to zero. We still need to honor all other timing parameters. However, we need to have an additional parameter which

Show more

Agreed. Revised parameters:

Parameter	Default	Units	Defines
`wakeHalfPressHoldTime`	10	sec	HP hold after wide PIR wake pulse
`minHalfPressBeforeShutter`	0.5	sec	Required HP pre-focus time before first shutter
`shutterPulseDuration`	100–250	ms	Duration of each FP pulse to camera
`burstFrameCount`	4	frames	Number of frames added/started by each FP trigger event

Parameter	Default	Units	Defines
`maxFrames`	12?	frames	Hard cap on total frames fired during one continuous activity episode
`interFrameDelay`	1.0	sec	Time between shutter activations
`postBurstHalfPressHoldTime`	2.0	sec	HP hold after final scheduled frame
`olicy`			during an active burst
`halfPressDuringBurstPolicy`	`independent`	enum	Burst logic does not itself drop HP
`fullPressWithoutPriorHpPolicy`	`assertHpThenWait`	enum	If FP arrives cold, assert HP and wait `minHalfPressBeforeShutter` before first shot
`halfPressInputDebounce`	20–50	ms	HP input noise rejection
`fullPressInputDebounce`	10–30	ms	FP input noise rejection
`inputActivePolarity`	active-low	enum	Incoming trigger line active state
`outputDriveMode`	open-drain/opto	enum	Camera output behaves like switch closure

Strike:

Removed	Reason
`cameraDriveMode`	Not really an MCU behavior parameter. The requirement is simply: camera is configured externally in single-shot mode; MCU generates repeated FP pulses.

Revised burst behavior:

Condition	Behavior
FP received while idle	Start a new burst: `remainingFrames = burstFrameCount`, bounded by `maxFrames`
FP received on shot 4 of 6	Reset burst progress: `remainingFrames = burstFrameCount` again
FP received repeatedly	Keep resetting scheduled burst count, but do not exceed `maxFrames` total frames for the current activity episode
`maxFrames` reached	Ignore/suppress further FP-triggered burst extension until current activity episode ends
Burst completes	Hold HP for `postBurstHalfPressHoldTime`, then release HP unless a fresh wake/trigger requires it

I'd name the enum:

```
triggerDuringBurstPolicy = resetRemainingBurstFrames
```

Definition:

```
When a new full-press input arrives during an active burst sequence,
the MCU resets the remaining burst frame counter to burstFrameCount,
subject to maxFrames for the current activity episode.
Already-fired frames are counted toward maxFrames.
Timing constraints are still honored:
- no overlapping shutter pulses
- interFrameDelay still applies
- minHalfPressBeforeShutter still applies when HP has not been active long enough
```

Example:

```
burstFrameCount = 6
maxFrames = 12

Initial FP:
frame 1
frame 2
frame 3
frame 4

New FP arrives:
```

remainingFrames resets to 6

Then MCU may shoot:
frame 5
frame 6
frame 7
frame 8
frame 9
frame 10

Total = 10 frames

If more FP events keep arriving, total frames may continue only until maxFrames = 12.